

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

Solution:

[SQL Worksheet]*



```
1 CREATE TABLE Customers (  
2     CustomerID NUMBER PRIMARY KEY,  
3     Name VARCHAR2(50),  
4     Age NUMBER,  
5     Balance NUMBER,  
6     IsVIP VARCHAR2(5)  
7 );  
8  
9 CREATE TABLE Loans (  
10     LoanID NUMBER PRIMARY KEY,  
11     CustomerID NUMBER,  
12     InterestRate NUMBER,  
13     DueDate DATE,  
14     FOREIGN KEY (CustomerID)  
15         REFERENCES Customers(CustomerID)  
16 );
```

[SQL Worksheet]*



```
1 INSERT INTO Customers VALUES (1, 'Aarav', 62, 16500, 'N');  
2 INSERT INTO Customers VALUES (2, 'Priya', 42, 14200, 'N');  
3 INSERT INTO Customers VALUES (3, 'Dev', 67, 9500, 'N');  
4  
5 INSERT INTO Loans VALUES (201, 1, 8.5, SYSDATE + 10);  
6 INSERT INTO Loans VALUES (202, 2, 9.5, SYSDATE + 20);  
7 INSERT INTO Loans VALUES (203, 3, 10.0, SYSDATE + 35);  
8  
9 COMMIT;
```

SELECT * FROM Customers;

SELECT * FROM Loans;

	CUSTOMERID	NAME	AGE	BALANCE	ISVIP
1	1	Aarav	62	16500	N
2	2	Priya	42	14200	N
3	3	Ravi	67	8500	N

Scenario 1:

```
[ SQL Worksheet ]*

1  BEGIN
2      FOR cust_rec IN (
3          SELECT CustomerID
4          FROM Customers
5          WHERE Age > 60
6      ) LOOP
7          UPDATE Loans
8          SET InterestRate = InterestRate - 1
9          WHERE CustomerID = cust_rec.CustomerID;
10         DBMS_OUTPUT.PUT_LINE(
11             'Discount applied for Customer ID: ' ||
12             cust_rec.CustomerID
13         );
14     END LOOP;
15     COMMIT;
16 END;
```

Query result	Script output	DBMS output	Explain Plan	SQL history
SQL> BEGIN				
FOR cust_rec IN (				
SELECT CustomerID FROM Customers...				
Show more...				
Discount applied for Customer ID: 1				
Discount applied for Customer ID: 3				
PL/SQL procedure successfully completed.				

SELECT LoanID, CustomerID, InterestRate FROM Loans;

	Query result	Script output	DBMS output	Explain Plan	SQL history
	Download v Execution time: 0.012 seconds				
	LOANID	CUSTOMERID	INTERESTRATE		
1	201	1	7.5		
2	202	2	9.5		
3	203	3	8.0		

Scenario 2:

```
[ SQL Worksheet ]* 
```

```
1  BEGIN
2      FOR cust_rec IN (
3          SELECT CustomerID
4          FROM Customers
5          WHERE Balance > 10000
6      ) LOOP
7          UPDATE Customers
8          SET IsVIP = 'TRUE'
9          WHERE CustomerID = cust_rec.CustomerID;
10         DBMS_OUTPUT.PUT_LINE(
11             'Customer promoted to VIP: ' ||
12             cust_rec.CustomerID
13         );
14     END LOOP;
15     COMMIT;
16 END;
```

Query resultScript outputDBMS outputExplain PlanSQL history

SQL> BEGIN
FOR cust_rec IN (
SELECT CustomerID FROM Customers...
Show more...

Customer promoted to VIP: 1
Customer promoted to VIP: 2

PL/SQL procedure successfully completed.

SELECT CustomerID, Name, Balance, IsVIP FROM Customers;

Query resultScript outputDBMS outputExplain PlanSQL history

Download v Execution time: 0.012 seconds

	CUSTOMERID	NAME	BALANCE	ISVIP
1	1	Aarav	16500	TRUE
2	2	Priya	14200	TRUE
3	3	Ravi	9500	FALSE

Scenario 3:

[SQL Worksheet]*



```
1 BEGIN
2   FOR loan_rec IN (
3     SELECT c.Name,
4            l.LoanID,
5            l.DueDate
6     FROM Customers c
7     JOIN Loans l ON c.CustomerID = l.CustomerID
8     WHERE l.DueDate BETWEEN SYSDATE
9            AND SYSDATE + 30
10  ) LOOP
11    DBMS_OUTPUT.PUT_LINE(
12      'Reminder: Dear ' || loan_rec.Name ||
13      ', your Loan ID ' || loan_rec.LoanID ||
14      ' is due on ' ||
15      TO_CHAR(loan_rec.DueDate, 'DD-MON-YYYY')
16    );
17  END LOOP;
```

Query result Script output DBMS output Explain Plan SQL history

```
SQL> BEGIN
      FOR loan_rec IN (
        SELECT c.Name, l.LoanID...
      Show more...

Reminder: Dear Aarav, your Loan ID 201 is due on 03-AUG-2026
Reminder: Dear Priya, your Loan ID 202 is due on 13-AUG-2026

PL/SQL procedure successfully completed.
```

SELECT c.Name, l.LoanID, l.DueDate FROM Customers c JOIN Loans l ON c.CustomerID = l.CustomerID
WHERE l.DueDate BETWEEN SYSDATE AND SYSDATE + 30;

Query result	Script output	DBMS output	Explain Plan	SQL history
Download	v	Execution time: 0.012 seconds		
	NAME	LOANID	DUE DATE	
1	Aarav	201	8/3/2026, 11:28:5	
2	Priya	202	8/13/2026, 11:28:5	

All outputs:

Query result	Script output	DBMS output	Explain Plan	SQL history
	Discount applied for Customer ID: 1 Discount applied for Customer ID: 3			
	Customer promoted to VIP: 1 Customer promoted to VIP: 2			
	Reminder: Dear Aarav, your Loan ID 201 is due on 03-AUG-2026 Reminder: Dear Priya, your Loan ID 202 is due on 13-AUG-2026			

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure `ProcessMonthlyInterest` that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure `UpdateEmployeeBonus` that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure `TransferFunds` that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

Solution:

[SQL Worksheet]*



```
1 CREATE TABLE Accounts (  
2     AccountID NUMBER PRIMARY KEY,  
3     CustomerID NUMBER,  
4     AccountType VARCHAR2(20),  
5     Balance NUMBER  
6 );
```

[SQL Worksheet]*



```
1 INSERT INTO Accounts VALUES (501, 201, 'SAVINGS', 12500);  
2 INSERT INTO Accounts VALUES (502, 202, 'SAVINGS', 7500);  
3 INSERT INTO Accounts VALUES (503, 203, 'CURRENT', 25000);  
4  
5 COMMIT;
```

`SELECT * FROM Accounts;`

Query result Script output DBMS output Explain Plan SQL history

Download v Execution time: 0.012 seconds

	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE
1	501	201	SAVINGS	12500
2	502	202	SAVINGS	7500
3	503	203	CURRENT	25000

Scenario 1:

[SQL Worksheet]*



```
1 CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
2 AS
3 BEGIN
4     UPDATE Accounts
5     SET Balance = Balance + (Balance * 0.01)
6     WHERE AccountType = 'SAVINGS';
7 COMMIT;
8 END;
```

SELECT object_name, object_type FROM user_objects WHERE object_name = 'PROCESSMONTHLYINTEREST';

Query result Script output DBMS output Explain Plan SQL history

Download v Execution time: 0.012 seconds

	OBJECT_NAME	OBJECT_TYPE
1	PROCESSMONTHLYINTEREST	PROCEDURE

BEGIN ProcessMonthlyInterest; END; /

Query result Script output DBMS output Explain Plan SQL history

```
SQL> BEGIN
      ProcessMonthlyInterest;
    END;
    /
```

PL/SQL procedure successfully completed.

SELECT * FROM Accounts;

	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE
1	501	201	SAVINGS	12625
2	502	202	SAVINGS	7575
3	503	203	CURRENT	35000

Scenario 2:

```
[ SQL Worksheet ]*

1 CREATE TABLE Employees (
2     EmployeeID NUMBER PRIMARY KEY,
3     EmployeeName VARCHAR2(50),
4     DepartmentID NUMBER,
5     Salary NUMBER
6 );
7
8 INSERT INTO Employees VALUES (10, 'Karan', 201, 45000);
9 INSERT INTO Employees VALUES (11, 'Meera', 201, 52000);
10 INSERT INTO Employees VALUES (12, 'Rahul', 202, 48000);
11
```

SELECT * FROM Employees;

	EMPLOYEEID	EMPLOYEENAME	DEPARTMENTID	SALARY
1	10	Karan	201	45000
2	11	Meera	201	52000
3	12	Rahul	202	48000

CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus...

[SQL Worksheet]*



```
1 CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(  
2     p_DepartmentID IN NUMBER,  
3     p_BonusPercent IN NUMBER  
4 ) AS  
5 BEGIN  
6     UPDATE Employees  
7     SET Salary = Salary + (Salary * p_BonusPercent / 100)  
8     WHERE DepartmentID = p_DepartmentID;  
9     COMMIT;  
10 END;
```

BEGIN UpdateEmployeeBonus(201, 10); END; /

Query result Script output DBMS output Explain Plan SQL history

```
SQL> BEGIN  
      UpdateEmployeeBonus(201, 10);  
      END;  
      /
```

PL/SQL procedure successfully completed.

SELECT * FROM Employees;

Query result Script output DBMS output Explain Plan SQL history

Download v Execution time: 0.012 seconds

	EMPLOYEEID	EMPLOYEENAME	DEPARTMENTID	SALARY
1	10	Karan	201	49500
2	11	Meera	201	57200
3	12	Ravi	202	48000

Scenario 3:

[SQL Worksheet]*



```
1 CREATE OR REPLACE PROCEDURE TransferFunds(  
2     p_FromAccount IN NUMBER,  
3     p_ToAccount   IN NUMBER,  
4     p_Amount      IN NUMBER  
5 ) AS  
6     v_Balance NUMBER;  
7 BEGIN  
8     SELECT Balance INTO v_Balance FROM Accounts  
9     WHERE AccountID = p_FromAccount;  
10    IF v_Balance >= p_Amount THEN  
11        UPDATE Accounts SET Balance = Balance - p_Amount  
12        WHERE AccountID = p_FromAccount;  
13        UPDATE Accounts SET Balance = Balance + p_Amount  
14        WHERE AccountID = p_ToAccount;  
15        COMMIT;  
16        DBMS_OUTPUT.PUT_LINE('Transfer Successful');  
17    ELSE  
18        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');  
19    END IF;  
20 EXCEPTION
```

BEGIN TransferFunds(501, 502, 2000); END; /

Query result Script output DBMS output Explain Plan SQL history

```
SQL> BEGIN  
      TransferFunds(501, 502, 2000);  
      END;  
      /
```

Transfer Successful

SELECT AccountID, Balance FROM Accounts;

	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE
1	501	201	SAVINGS	10625
2	502	202	SAVINGS	9575
3	503	203	CURRENT	35000

BEGIN TransferFunds(501, 502, 50000); END; /

	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE
1	501	201	SAVINGS	10625
2	502	202	SAVINGS	9575
3	503	203	CURRENT	35000

SQL> BEGIN
 TransferFunds(501, 502, 50000);
END;
/

Insufficient Balance

SELECT AccountID, Balance FROM Accounts;

	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE
1	501	201	SAVINGS	10625
2	502	202	SAVINGS	9575
3	503	203	CURRENT	35000